МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к лабораторной работе № 2

«Создание экспертной системы с использованием обратной цепочки рассуждений»

Студент ПИ-23-1 Белозерцев Д.Р.

группа подпись, дата фамилия, инициалы

Руководитель

Первушин О.С.

ученая степень, ученое звание подпись, дата фамилия, инициалы

Липецк 2026 г.

Введение

Экспертные системы являются одним из направлений искусственного интеллекта, в котором знания предметной области представляются в виде фактов и правил, а специальный механизм вывода позволяет получать новые знания на основании уже известных. Одним из вариантов такого вывода является обратная цепочка рассуждений.

Целью данной части лабораторной работы является дополнение ранее разработанной программной оболочки механизмом обратной цепочки рассуждений без удаления реализованного механизма прямой цепочки.

В качестве предметной области сохраняется формирование списка покупок для приготовления блюда. В отличие от прямой цепочки, при обратном выводе пользователь задаёт конкретную целевую ситуацию, после чего программа пытается доказать её достижимость, двигаясь от цели к исходным фактам.

В ходе работы был разработан отдельный модуль backward_inference.py, реализующий поиск правил, способных привести к заданной цели, рекурсивное доказательство условий этих правил, обработку уже доказанных фактов и защиту от циклических зависимостей.

1 Постановка задачи

Необходимо дополнить разработанную на языке Python оболочку экспертной системы механизмом обратной цепочки рассуждений. Существующий модуль прямой цепочки должен сохраниться и продолжить функционировать независимо от нового механизма.

Система должна обеспечивать следующие возможности:

загрузка базы правил из отдельного текстового файла;

задание начального состояния рабочей базы данных;

задание целевой ситуации;

поиск правил, заключения которых совпадают с целевой ситуацией;

рекурсивное доказательство условий найденных правил;

определение достижимости целевой ситуации;

вывод последовательности доказательства на экран;

обработку ситуации, когда цель не может быть доказана;

защиту от циклических зависимостей между фактами;

повторное использование фактов, которые уже были доказаны;

сохранение возможности запуска ранее реализованной прямой цепочки.

2 Описание предметной области

2.1 Выбор предметной области

Предметной областью является формирование списка покупок для приготовления блюда. Пользователь задаёт требования к блюду и приёму пищи. Эти требования представлены исходными фактами рабочей базы данных.

База правил описывает зависимости между требованиями, промежуточными признаками и необходимыми продуктами. Например, если требуется быстрый ужин, система может определить подходящий простой вариант блюда, затем подходящий продукт и в конечном результате подтвердить необходимость покупки конкретного продукта.

2.2 Принцип работы экспертной системы с обратной цепочкой

При прямой цепочке система начинает с исходных фактов и последовательно получает новые факты. При обратной цепочке направление рассуждения изменяется: система начинает с заданной пользователем цели и ищет правила, которые способны привести к этой цели.

После выбора подходящего правила его условия становятся подцелями. Каждая подцель проверяется аналогичным образом. Если условие уже присутствует в рабочей базе данных, оно считается доказанным. Если условие отсутствует, система пытается получить его с помощью другого правила.

Доказательство завершается успешно, если для некоторого правила доказаны все его условия. Если для цели отсутствуют подходящие правила или хотя бы одно необходимое условие невозможно доказать, целевая ситуация считается недостижимой.

3 Проектирование экспертной системы

3.1 Архитектура системы

Для сохранения существующего механизма прямого вывода новый алгоритм реализован отдельным модулем. Общая структура проекта имеет следующий вид:

```
project/
├── main.py
├── rules_manager.py
├── inference.py
├── backward_inference.py
└── rules.txt
```

Модуль main.py отвечает за главное меню, ввод начальной ситуации и целевой ситуации, отображение данных и запуск алгоритмов вывода.

Модуль rules_manager.py содержит класс Rule, функции загрузки и сохранения базы правил, проверки корректности и CRUD-операции.

Модуль inference.py содержит ранее реализованный механизм прямой цепочки рассуждений.

Модуль backward_inference.py содержит механизм обратной цепочки. Он использует те же объекты Rule и ту же базу правил, но применяет их в обратном направлении — от заключения к условиям.

Файл rules.txt является внешней базой знаний и не зависит от конкретного алгоритма вывода.

3.2 Представление правил

Формат правил остаётся неизменным. Каждая строка содержит идентификатор, набор условий и заключение:

R01 | нужен_ужин & нужно_быстрое_блюдо | подходит_простое_блюдо

При прямом выводе правило рассматривается как переход от условий к заключению. При обратном выводе его заключение рассматривается как цель, для достижения которой необходимо доказать все условия.

A & B → C прямая цепочка: A и B позволяют получить C

C → A, B обратная цепочка: для доказательства C необходимо доказать A и B

3.3 Начальное состояние рабочей базы данных

Перед запуском обратной цепочки пользователь задаёт начальное состояние рабочей базы данных. Для демонстрационного сценария используются следующие факты:

- нужен_ужин
- нужно_быстрое_блюдо
- нужно_мясное_блюдо
- нужно_недорогое_блюдо

3.4 Целевая ситуация

В отличие от прямой цепочки, при обратном выводе необходимо явно задать цель. В демонстрационном сценарии целевой ситуацией является:

```
купить_куриную_грудку
```

Система должна определить, может ли эта цель быть доказана на основании исходных фактов и правил.

3.5 Алгоритм доказательства цели

Основной алгоритм реализован рекурсивно. Сначала проверяется, является ли цель уже известным фактом. Если нет, находятся правила, заключение которых совпадает с целью. Затем последовательно доказываются все условия подходящего правила.

```
если цель уже есть в РБД:
    цель доказана
иначе:
    найти правила, выводящие цель
    для каждого правила:
        доказать все его условия
        если все условия доказаны:
            цель доказана
    если подходящего доказательства нет:
        цель не доказана
```

Для предотвращения ошибочного определения циклов используются два множества. visited содержит цели, находящиеся в текущем пути доказательства, а proved — факты, которые уже были успешно доказаны. Такое разделение позволяет повторно использовать ранее доказанный факт.

4 Разработка программной части

4.1 Структура программы

Разработка выполнена с сохранением существующих модулей. Добавление обратной цепочки не изменяет алгоритм прямого вывода и не требует изменения формата файла rules.txt.

В main.py добавлен импорт функции backward_chain и отдельный пункт меню для запуска обратного вывода. Пользователь сначала задаёт исходную рабочую базу данных, затем указывает цель.

4.2 Модуль управления правилами

Модуль rules_manager.py используется обоими алгоритмами вывода. Он загружает правила из файла, проверяет их корректность и предоставляет операции добавления, изменения и удаления. Таким образом, база знаний является общей для прямой и обратной цепочек.

4.3 Модуль обратной цепочки рассуждений

В модуле backward_inference.py реализованы три основные функции: find_rules_for_goal(), prove_goal() и backward_chain().

Функция find_rules_for_goal() перебирает базу правил и возвращает правила, заключение которых совпадает с заданной целью.

Функция prove_goal() рекурсивно доказывает цель. При наличии нескольких условий правила каждое условие рассматривается как отдельная подцель.

Функция backward_chain() является внешней точкой запуска алгоритма. Она создаёт структуры для контроля циклов и уже доказанных фактов, вызывает рекурсивное доказательство и выводит итог: цель доказана или цель не доказана.

```
def find_rules_for_goal(rules, goal):
    result = []

    for rule in rules:
        if rule.conclusion == goal:
            result.append(rule)

    return result

def backward_chain(rules, working_memory, goal):
    visited = set()
```

```
proved = set()

result = prove_goal(
    rules,
    working_memory,
    goal,
    visited,
    proved,
    level=0
)

return result
```

4.4 Главное меню и взаимодействие с пользователем

В главное меню добавлен пункт запуска обратной цепочки:

```
1. Задать начальную ситуацию
2. Показать рабочую базу данных
3. Запустить прямую цепочку рассуждений
4. Запустить обратную цепочку рассуждений
5. Показать базу правил
6. Добавить правило
7. Изменить правило
8. Удалить правило
9. Очистить рабочую базу данных
0. Выход
```

При выборе пункта 4 программа запрашивает целевую ситуацию и передаёт её в функцию backward_chain(). Начальная рабочая база данных при этом используется как набор исходных фактов.

5 Реализация обратной цепочки рассуждений

5.1 Поиск правил для цели

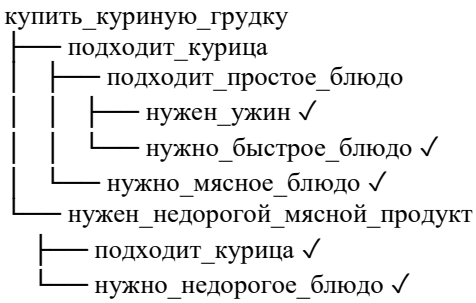
Для каждой заданной цели система сначала ищет правила, заключение которых совпадает с этой целью. Например, для цели «купить_куриную_грудку» находится правило R04:

```
R04 | подходит_курица & нужен_недорогой_мясной_продукт | купить_куриную_грудку
```

После выбора R04 его условия становятся подцелями: «подходит_курица» и «нужен_недорогой_мясной_продукт».

5.2 Рекурсивное доказательство подцелей

Для доказательства «подходит_курица» находится правило R02. Для доказательства «подходит_простое_блюдо» используется R01. Условия R01 «нужен_ужин» и «нужно_быстрое_блюдо» находятся непосредственно в начальной рабочей базе данных и поэтому считаются доказанными.



Таким образом, система строит дерево доказательства от целевой ситуации к исходным фактам.

5.3 Защита от циклов и повторное использование доказанных фактов

В процессе доказательства одна и та же подцель может встретиться несколько раз. Например, факт «подходит_курица» используется при доказательстве двух разных ветвей цели. Если считать любой повторный вызов этой подцели циклом, достижимая цель ошибочно будет признана недоказанной.

Для устранения этой проблемы введено множество proved. После успешного доказательства факта он помещается в proved и при следующем обращении считается уже доказанным. Множество visited используется только для обнаружения настоящего цикла в текущем пути рекурсии.

6 Тестирование программы

6.1 Проверка достижимой цели

В качестве начальной рабочей базы использованы факты «нужен_ужин», «нужно_быстрое_блюдо», «нужно_мясное_блюдо» и «нужно_недорогое_блюдо». Цель — «купить_куриную_грудку».

В процессе работы система находит R04, затем доказывает его условия через R02 и R03. В результате цель успешно доказывается.

```
Цель: купить_куриную_грудку
...
✓ Правило R04 доказывает купить_куриную_грудку

ЦЕЛЬ ДОКАЗАНА.
```

6.2 Проверка недостижимой цели

Для проверки отрицательного результата рабочая база была изменена так, чтобы в ней отсутствовали факты, необходимые для получения «подходит_макароны». Целью была задана «купить_макароны».

Для этой цели используется R15, но для его выполнения необходимо доказать «подходит_макароны». Оно получается только через R14, требующее «нужен_обед» и «нужно_вегетарианское_блюдо». При отсутствии этих фактов и правил для их получения доказательство прекращается.

```
✗ Не удалось доказать подходит_макароны
✗ Не удалось доказать купить_макароны

ЦЕЛЬ НЕ ДОКАЗАНА.
```

6.3 Проверка цели, уже содержащейся в РБД

В рабочую базу данных непосредственно добавлялся факт «купить_куриную_грудку», после чего этот же факт задавался в качестве цели. Система должна немедленно признать цель доказанной без поиска дополнительных правил.

6.4 Проверка цели без соответствующего правила

В качестве цели задавался факт «купить_шоколад», для которого отсутствует правило с соответствующим заключением. Система сообщает об отсутствии правила, способного получить данную цель, и завершает доказательство отрицательным результатом.

6.5 Проверка совместной работы прямой и обратной цепочек

После добавления нового модуля отдельно запускался существующий механизм прямой цепочки. Он продолжает использовать inference.py и формировать новые факты в рабочей базе данных. Затем можно запустить обратную цепочку с выбранной целью. Таким образом, оба алгоритма работают независимо и используют общую базу правил.

Заключение

В ходе работы разработан механизм обратной цепочки рассуждений для ранее созданной экспертной системы формирования списка покупок. При этом существующий модуль прямой цепочки сохранён, а новый алгоритм вынесен в отдельный модуль backward_inference.py.

Реализованный механизм начинает работу с заданной целевой ситуации, ищет правила, способные привести к ней, и рекурсивно доказывает условия найденных правил. Если все необходимые условия подтверждены исходными фактами или другими правилами, цель считается достижимой. В противном случае система сообщает, что цель не доказана.

Дополнительно была реализована защита от циклических зависимостей и механизм повторного использования уже доказанных фактов. Проведённые тесты подтвердили возможность доказательства достижимых целей и корректную обработку недостижимых целей.

Таким образом, программная оболочка поддерживает два независимых механизма вывода — прямую и обратную цепочки рассуждений, что позволяет исследовать два противоположных способа построения логического вывода на одной базе знаний.

ПРИЛОЖЕНИЕ А — ЛИСТИНГ МОДУЛЯ ОБРАТНОЙ ЦЕПОЧКИ

```
def find_rules_for_goal(rules, goal):
    result = []

    for rule in rules:
        if rule.conclusion == goal:
            result.append(rule)

    return result


def backward_chain(rules, working_memory, goal):
    print("\n=== ОБРАТНАЯ ЦЕПОЧКА РАССУЖДЕНИЙ ===")
    print(f"Цель: {goal}\n")

    visited = set()
    proved = set()

    result = prove_goal(
        rules,
        working_memory,
        goal,
        visited,
        proved,
        level=0
    )

    if result:
        print("\nЦЕЛЬ ДОКАЗАНА.")
        return True

    print("\nЦЕЛЬ НЕ ДОКАЗАНА.")
    return False


def prove_goal(
    rules,
    working_memory,
    goal,
    visited,
    proved,
    level=0
):
    indent = "  " * level

    if goal in working_memory:
        print(f"{indent}✓ {goal} — факт известен")
        proved.add(goal)
        return True

    if goal in proved:
        print(f"{indent}✓ {goal} — уже доказан")
        return True

    if goal in visited:
        print(f"{indent}✗ {goal} — обнаружен цикл")
```

```
    return False

print(f"{indent}→ Требуется доказать: {goal}")
visited.add(goal)

matching_rules = find_rules_for_goal(rules, goal)

if not matching_rules:
    print(
        f"{indent}✗ Нет правила, "
        f"которое выводит {goal}"
    )
    visited.remove(goal)
    return False

for rule in matching_rules:
    print(
        f"{indent}Проверяем правило "
        f"{rule.rule_id}"
    )

    all_conditions_proved = True

    for condition in rule.conditions:
        if not prove_goal(
            rules,
            working_memory,
            condition,
            visited,
            proved,
            level + 1
        ):
            all_conditions_proved = False
            break

    if all_conditions_proved:
        print(
            f"{indent}✓ Правило {rule.rule_id} "
            f"доказывает {goal}"
        )
        proved.add(goal)
        visited.remove(goal)
        return True

print(
    f"{indent}✗ Не удалось доказать {goal}"
)

visited.remove(goal)
return False
```

ПРИЛОЖЕНИЕ Б — ФАЙЛ БАЗЫ ПРАВИЛ

```
# База правил экспертной системы
# Формат: ID | УСЛОВИЯ | ВЫВОД

R01 | нужен_ужин & нужно_быстрое_блюдо | подходит_простое_блюдо
R02 | подходит_простое_блюдо & нужно_мясное_блюдо | подходит_курица
R03 | подходит_курица & нужно_недорогое_блюдо | нужен_недорогой_мясной_продукт
R04 | подходит_курица & нужен_недорогой_мясной_продукт | купить_куриную_грудку

R05 | подходит_простое_блюдо | нужен_гарнир
R06 | нужен_гарнир & нужно_недорогое_блюдо | подходит_рис
R07 | подходит_рис | купить_рис

R08 | нужен_ужин & нужно_полезное_блюдо | нужны_овощи
R09 | нужны_овощи | купить_овощи

R10 | нужен_завтрак & нужно_быстрое_блюдо | нужны_яйца
R11 | нужны_яйца | купить_яйца

R12 | нужен_завтрак & нужно_полезное_блюдо | нужны_фрукты
R13 | нужны_фрукты | купить_фрукты

R14 | нужен_обед & нужно_вегетарианское_блюдо | подходит_макароны
R15 | подходит_макароны | купить_макароны
```

ПРИЛОЖЕНИЕ В — ДОКУМЕНТАЦИЯ ПРОМПТОВ И ОТВЕТОВ

запрос: «как реализовать обратную цепочку рассуждений, не удаляя модуль для прямой? и как надо составить для этого базу правил. необходимо также задать начальное состояние рабочей базы данных, затем - целевую ситуацию, потом с помощью обратной цепочки рассуждений доказать или опровергнуть возможность достижения заданной целевой ситуации.»

ответ: Была предложена архитектура с сохранением inference.py и добавлением отдельного backward_inference.py. Определено, что база правил может быть общей для обоих алгоритмов. Для обратной цепочки начальная РБД и целевая ситуация задаются отдельно, а алгоритм движется от цели к исходным фактам.

запрос: «как добавить в main.py новый модуль»

ответ: Показано, как импортировать backward_chain из backward_inference.py, добавить пункт меню и передавать в функцию правила, рабочую базу данных и целевую ситуацию.

запрос: «как протестировать»

ответ: Был предложен набор тестов: достижимая цель, недостижимая цель, цель уже в рабочей базе данных, цель без соответствующего правила и проверка того, что после добавления обратной цепочки продолжает работать прямая цепочка.

запрос: сообщение об ошибке TypeError: backward_chain() missing 1 required positional argument: 'goal'

ответ: Ошибка возникла из-за вызова backward_chain() с двумя аргументами вместо трёх. В main.py был добавлен ввод цели и передача goal третьим аргументом.

запрос: вывод программы, в котором «подходит_курица — обнаружен цикл» и цель ошибочно признаётся недоказанной

ответ: Ошибка была связана с тем, что уже доказанный факт ошибочно воспринимался как цикл. Алгоритм был исправлен разделением visited для текущего пути доказательства и proved для уже успешно доказанных фактов.